

Checkout Intelligence Agent

Technical Design Document

Team Seven Up · Version 1.0 · 2026

Document Type	Technical Design Document (TDD)
Project	Checkout Intelligence Agent
Team	Seven Up
Status	In Development
Version	1.0
Date	2026

1. System Overview

The Checkout Intelligence Agent is a full-stack web application built on a Node.js/Express REST API backend and a React/Vite frontend. It integrates OpenAI's GPT-4o-mini for AI reasoning and Shopify's Admin GraphQL API for product catalog synchronization. Session state and behavioral event logs are persisted in MongoDB Atlas.

Layer	Technology	Responsibility
Frontend	React 19, Vite 8, Tailwind CSS 4	Store UI, Checkout flow, KasparAI chat widget
State Management	React Context (CartContext, AuthContext)	Cart state, session ID, auth tokens
API Client	Axios 1.x	HTTP calls to backend REST API
Animation	Framer Motion 12	Widget open/close, step transitions, micro-animations
Icons	Lucide React	Consistent icon set across all components
Backend	Node.js + Express 5	REST API, session management, event processing
AI Reasoning	OpenAI SDK (gpt-4o-mini)	Hesitation classification, conversational recovery
Database	MongoDB + Mongoose 9	Session persistence, event log storage
E-commerce	Shopify Admin GraphQL API	Product catalog fetch with mock data fallback
Build Tool	Vite 8 (frontend)	HMR, bundling, env variable injection
Dev Server	Nodemon 3	Auto-restart on file changes during development

2. Repository Structure

```
checkout-intelligence-agent/  
  ■■■■ backend/  
    ■ ■■■■ agents/  
      ■ ■ ■■■ CheckoutAgent.js # Hesitation analysis + persuasion copy  
      ■ ■ ■■■ PricingAgent.js # Discount generation + coupon validation  
      ■ ■ ■■■ ShoppingAgent.js # Product recommendations + search  
    ■ ■■■■ models/  
      ■ ■ ■■■ Session.js # Mongoose session schema  
      ■ ■ ■■■ EventLog.js # Behavioral event schema  
    ■ ■■■■ utils/  
      ■ ■ ■■■ analyzer.js # GPT-4o-mini hesitation classifier  
      ■ ■ ■■■ conversation.js # Multi-turn recovery conversation engine  
      ■ ■ ■■■ shopify.js # Shopify GraphQL client + mock catalog
```

```
■ ■■■ server.js # Express app + all API routes
■ ■■■ package.json
■ ■■■ .env.template
■■■ frontend/
■■■ src/
■ ■■■ components/layout/ # Navbar
■ ■■■ context/ # CartContext, AuthContext
■ ■■■ hooks/
■ ■ ■■■ useCheckoutTracker.js # Behavioral event tracker hook
■ ■■■ pages/
■ ■■■ Home.jsx, Login.jsx, Signup.jsx
■ ■■■ Store.jsx # Product catalog page
■ ■■■ Checkout.jsx # Full checkout + KasparAI widget
■ ■■■ Dashboard.jsx # Analytics dashboard
■ ■■■ Demo.jsx
■■■ index.html
■■■ vite.config.js
■■■ package.json
```

3. Backend API Reference

Method	Endpoint	Description	Key Request Body Fields
GET	/api/products	Fetch product catalog (Shopify or mock)	—
POST	/api/sessions	Create or update a checkout session	sessionId, cartItems, cartTotal
POST	/api/events	Log a behavioral event; triggers friction analysis	sessionId, eventType, metadata
GET	/api/intervention/:sessionId	Poll whether an intervention has been triggered	—
POST	/api/chat	Send a user message to KasparAI recovery agent	sessionId, userMessage
POST	/api/sessions/:sessionId/complete	Mark session as checkout-complete	—
GET	/api/analytics	Retrieve aggregated recovery analytics	—

4. Data Models

4.1 Session Schema (MongoDB)

Field	Type	Default	Description
sessionId	String	required	UUID uniquely identifying a checkout session
checkoutStage	String	'cart'	cart checkout payment completed
cartItems	Array	[]	Snapshot of cart at time of last update
cartTotal	Number	0	Computed cart subtotal in USD
frictionScore	Number	0	Accumulated behavioral friction (0–100)
interventionTriggered	Boolean	false	Whether KasparAI has been activated
interventionType	String	null	Classified friction reason
conversationHistory	Array	[]	Full {{role, content}} chat history
recoveredAt	Date	null	Timestamp of checkout completion post-intervention
createdAt / updatedAt	Date	now	Auto-managed timestamps

4.2 EventLog Schema (MongoDB)

Field	Type	Description
sessionId	String	Reference to the parent Session document
eventType	String	idle_checkout cart_modified shipping_method_changed address_edited coupon_failed exit_intent

metadata	Mixed	Arbitrary event-specific data (e.g. { section: 'payment', field: 'card_number' })
timestamp	Date	Auto-set on document creation

5. Agent Architecture

The backend implements a lightweight multi-agent orchestration pattern. Three specialized agents are instantiated as ES6 classes and called selectively by the conversation utility based on the user message content and checkout context.

CheckoutAgent

- `analyzeHesitation(session, events)`: Inspects last 5 events for navigation patterns; detects idle on payment section.
- `persuade(session)`: Returns a randomly selected urgency/social-proof message to encourage checkout completion.
- Activated when `session.checkoutStage === 'started'` and no other intent keyword is detected.

PricingAgent

- `generateOffer(session)`: Returns a discount object based on `cartTotal` thresholds — `WELCOME15` for carts > \$500, `SAVE10` otherwise.
- `validateCoupon(code)`: Looks up code in a static map; returns `{ discount, type }` or `null`.
- Activated when user message contains `price`, `discount`, or `offer` keywords.

ShoppingAgent

- `constructor(products)`: Initialized with the full product catalog (fetched from Shopify or mock).
- `recommend(session, history)`: Filters catalog by categories matching cart items; returns top 3 not already in cart.
- `search(query)`: Full-text search across title, description, and category fields.
- Activated when user message contains `recommend`, `looking for`, or `find` keywords.

6. AI Integration

6.1 Hesitation Classifier (`analyzer.js`)

Sends session context (`cartTotal`, `cartItemCount`, `frictionScore`, event list) to GPT-4o-mini with a system prompt that constrains the response to a strict JSON object: `{ reason, confidence, explanation }`. Only reasons with `confidence > 0.55` trigger an intervention; `reason = 'other'` is always filtered out. Falls back to deterministic rule-based logic when the API is unavailable.

6.2 Conversational Recovery Engine (conversation.js)

Maintains a full conversation history array in the Session document. On each /api/chat request, the history is prepended as prior turns in the GPT-4o-mini messages array. The system prompt includes the active friction type, cart context, and any agent insight generated by the specialized sub-agents. The model is instructed to emit a [RESOLVED:] token upon successful resolution. max_tokens is set to 300 to enforce brevity.

6.3 Fallback Behavior

When OPENAI_API_KEY is missing or set to the placeholder value, both the classifier and conversation engine fall back to deterministic rule-based logic. The fallback classifier counts event type frequencies and maps them to friction reasons. The fallback conversation engine pattern-matches user message keywords and returns pre-written offer replies with [RESOLVED] tokens.

7. Frontend Architecture

7.1 Application Routing

Route	Component	Description
/	Home.jsx	Landing page with hero section and CTA
/store	Store.jsx	Product catalog with filters, cart management, product hover tracking
/checkout	Checkout.jsx	Full checkout flow with KasparAI widget, friction tracking, offer application
/dashboard	Dashboard.jsx	Analytics view with mock KPI cards and revenue chart
/login	Login.jsx	Auth form (UI prototype)
/signup	Signup.jsx	Registration form (UI prototype)
/demo	Demo.jsx	Agent demonstration page

7.2 State Management

CartContext provides: cart (array), sessionId (uuid), addToCart, removeFromCart, updateQuantity, clearCart, getCartTotal. The sessionId is generated once per app mount and persisted to localStorage, ensuring session continuity across page refreshes within the same browser tab.

7.3 useCheckoutTracker Hook

A custom React hook that encapsulates all behavioral telemetry. It registers activity listeners (mousemove, keypress, scroll, click) to detect idle periods and fires idle_checkout events every 10 seconds of inactivity. A separate 10-second proactive interval (independent of user activity) fires the onProactiveTick callback for agent-initiated messages. Section dwell timing is measured per named section with a minimum threshold of 8 seconds before logging a section_dwell event.

7.4 KasparAI Widget State Machine

State	Condition	UI
Hidden	No intervention triggered, not proactively opened	Nothing visible
Minimized	agentVisible=true, chatOpen=false	Floating header bar with bot icon + unread badge
Open	agentVisible=true, chatOpen=true	Full chat panel with messages, quick replies, input
Dismissed	User clicks X button	Hidden; re-opens on next trigger

8. Environment Configuration

Backend (.env)

```
MONGODB_URI=mongodb://127.0.0.1:27017/checkout-recovery
OPENAI_API_KEY=your_openai_api_key # Optional – enables GPT-4o-mini
SHOPIFY_STORE_DOMAIN=your-store.myshopify.com
SHOPIFY_ADMIN_ACCESS_TOKEN=shpat_XXXXX
PORT=5000
```

Frontend (.env)

```
VITE_API_URL=http://localhost:5000/api
```

9. Friction Scoring Logic

Event Type	Score Increment	Rationale
exit_intent	+30	Strong abandonment signal; immediate action required
coupon_failed	+25	Clear sign of deal-seeking frustration
idle_checkout	+20	Prolonged hesitation without progress
payment_error	+20	Technical barrier; high drop-off risk
shipping_method_changed	+15	Decision uncertainty about delivery cost/speed
price_hover	+15	Explicit price inspection behavior
address_edited	+10	Multiple address edits suggest distraction or error
cart_modified	+10	Cart changes indicate reconsideration
default (unknown)	+5	Any other tracked event adds minor friction

The score is capped at 100. When the AI classifier returns no confident result, the score is reset to 25 (a 'soft reset') rather than zero, maintaining elevated awareness without re-triggering immediately.

10. Deployment & Local Setup

- 1. Clone repository and install dependencies:

```
cd backend && npm install
cd frontend && npm install
```

- 2. Copy environment templates and fill in credentials:

```
backend/.env.template → backend/.env
```

```
frontend/.env.example → frontend/.env
```

3. Start MongoDB (local or Atlas connection string in MONGODB_URI).
4. Run backend: `cd backend && npm run dev` (port 5000)
5. Run frontend: `cd frontend && npm run dev` (Vite default port 5173)
6. Open `http://localhost:5173` — the store is live.

Note: All AI and Shopify features degrade gracefully if keys are missing.

11. Known Technical Debt & Limitations

- Unresolved git merge conflicts in `conversation.js` and `shopify.js` between the HEAD branch and a feature branch (3a7b050). Both versions are present in the file and must be resolved before production deployment.
- No authentication middleware on any backend route — the API is fully open. JWT or API key validation should be added before public deployment.
- `CartContext` uses `localStorage` for session persistence; this couples the frontend to the browser environment and must be replaced with server-side session tokens for multi-device support.
- The analytics dashboard uses static/mock data for chart visualization. It should be wired to the live `/api/analytics` endpoint with real time-series aggregation.
- No automated test suite exists (backend scripts list echo only). Unit tests for agents, integration tests for API routes, and E2E tests with Playwright are recommended.
- The Stripe SDK (`@stripe/stripe-js`) is installed as a frontend dependency but not yet integrated — payment fields are UI-only prototypes.
- OpenAI API calls have no retry logic or rate-limit handling beyond a basic catch block.